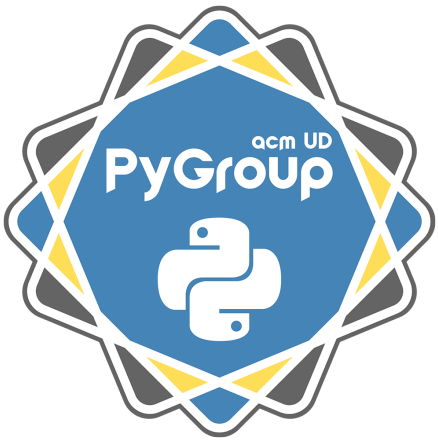


Python 0 a 100

Curso ACM

Proyecto Final

Índice

1. Contexto de la problemática **2**
1.1. Caso de estudio: Tienda “Don Aurelio” . . . 2
1.2. Reto del proyecto . . . 2

2. Objetivos **3**
2.1. Objetivo general . . . 3
2.2. Objetivos específicos . . . 3

3. Descripción funcional **3**
3.1. Modelo de datos . . . 3
3.2. Módulos funcionales . . . 3
3.3. Reglas de negocio . . . 4
3.4. Formato de persistencia . . . 4

4. Criterios de aceptación **5**
4.1. Criterios funcionales . . . 5
4.2. Criterios de robustez . . . 6

5. Estructura sugerida del repositorio **6**

6. Entregables **6**

1. Contexto de la problemática

En Colombia existen cerca de 450 000 tiendas de barrio, que concentran una parte muy importante del comercio minorista de productos de consumo masivo. Muchas de ellas son negocios familiares que todavía llevan el control de sus existencias y ventas en un cuaderno, en hojas sueltas o de memoria. Esta práctica genera problemas recurrentes que afectan directamente la rentabilidad del negocio.

1.1. Caso de estudio: Tienda "Don Aurelio"

Don Aurelio administra desde hace quince años una tienda de barrio en la localidad de Kennedy, en Bogotá. Atiende en promedio 80 clientes al día y maneja alrededor de 150 referencias de productos: abarrotes, lácteos, bebidas, aseo y snacks. Su hija, estudiante de Ingeniería de Sistemas, le ayudó a identificar las siguientes dificultades:

- **Desabastecimiento inesperado:** no sabe qué productos se están agotando hasta que un cliente los pide y no los encuentra.
- **Errores de cálculo:** los totales de las ventas se suman a mano, y los errores en el cambio y en el valor final son frecuentes.
- **Productos sin rotación:** no tiene claro qué referencias llevan semanas sin venderse y ocupan espacio y capital.
- **Pérdida de información:** el cuaderno de ventas se ha extraviado y mojado en más de una ocasión.
- **Compras ineficientes:** al pedir mercancía a los proveedores, no sabe cuánto ni a quién pedirle.
- **Sin cierre de caja:** al finalizar el día no puede verificar cuánto vendió realmente ni cuáles fueron los productos más vendidos.

1.2. Reto del proyecto

Su grupo ha sido contratado como equipo de desarrollo para construir **TiendaPy**, una aplicación de consola escrita en Python que permita a Don Aurelio registrar su inventario, procesar ventas, controlar el stock mínimo, conservar la información de forma persistente y generar reportes diarios. La aplicación debe ser lo bastante robusta para que una persona sin formación técnica pueda usarla sin que el programa se detenga por errores de digitación.

Alcance

El programa se ejecuta en consola, utiliza archivos de texto plano como mecanismo de persistencia y debe desarrollarse con las herramientas vistas en el curso. No se requieren bases de datos ni interfaces gráficas.

2. Objetivos

2.1. Objetivo general

Desarrollar una aplicación de consola en Python que solucione los problemas de control de inventario y ventas de una tienda de barrio, aplicando los conceptos fundamentales de programación y las buenas prácticas del ecosistema profesional.

2.2. Objetivos específicos

1. Modelar los productos, las ventas y los proveedores mediante estructuras de datos nativas adecuadas.
2. Implementar la lógica del negocio con funciones modulares, reutilizables y desacopladas.
3. Validar las entradas del usuario y manejar errores de forma controlada.
4. Persistir la información en archivos de texto plano para que no se pierda entre ejecuciones.
5. Gestionar el desarrollo con Git, documentarlo con un `README.md` y aislar el entorno con `venv`.

3. Descripción funcional

3.1. Modelo de datos

Cada producto del inventario se identifica por un código único y tiene los siguientes atributos:

Atributo	Tipo	Descripción
`codigo`	`str`	Identificador único (p. ej. `LAC001`)
`nombre`	`str`	Nombre comercial del producto
`categoria`	`str`	Lácteos, Bebidas, Aseo, Abarrotes, Snacks
`precio`	`float`	Precio de venta unitario en COP
`stock`	`int`	Unidades disponibles
`stock_minimo`	`int`	Umbral para generar alerta de reabastecimiento
`proveedor`	`str`	Nombre del proveedor habitual

Tabla 1: Atributos de un producto.

3.2. Módulos funcionales

M1. Gestión de inventario. Registrar, consultar, modificar y eliminar productos. Buscar por código o por categoría.

M2. Registro de ventas. Permitir al usuario construir una venta con varios productos (carrito), calcular subtotal, descuento y total, recibir el pago y calcular el cambio. Al confirmar, se descuenta el stock.

M3. Alertas de stock. Listar los productos cuyo stock sea menor o igual al stock mínimo, agrupados por proveedor, para facilitar el pedido de mercancía.

M4. Reportes. Generar el cierre del día: total vendido, número de transacciones, ticket promedio, producto más vendido, productos sin ventas y categorías con movimiento.

M5. Persistencia. Cargar el inventario al iniciar y guardarlo al cerrar o cuando el usuario lo solicite. Mantener un historial de ventas en un archivo aparte.

M6. Menú principal. Interfaz de consola que se mantiene activa hasta que el usuario decida salir.

3.3. Reglas de negocio

- No se puede vender una cantidad mayor al stock disponible.
- No se permiten precios ni cantidades negativas o iguales a cero.
- Las compras con un subtotal de \$50 000 COP o más reciben un descuento del 5 %; las de \$100 000 COP o más, un descuento del 10 %.
- El pago recibido no puede ser inferior al total a pagar.
- Los códigos de producto no pueden repetirse.
- Un producto con stock igual a cero debe aparecer en las alertas como *agotado*.

3.4. Formato de persistencia

El inventario se almacena en `data/inventario.txt`, un producto por línea, con campos separados por el carácter `"|"`:

```
LAC001|Leche entera 1L|Lacteos|4200.0|35|10|Alqueria
BEB003|Gaseosa 1.5L|Bebidas|5800.0|8|12|Postobon
ASE002|Detergente 500g|Aseo|7500.0|20|5|Distribuidora Norte
```

Las ventas se registran al final de `data/ventas.txt`, una venta por línea, con formato `fecha|total|codigo:cantidad;codigo:cantidad`:

```
2026-11-20|18400.0|LAC001:2;BEB003:1
```

Sobre la dependencia externa

Se recomienda utilizar la librería `tabulate` para mostrar las tablas de inventario y reportes en consola. El grupo puede proponer otra librería, siempre que se justifique su uso y se declare en `requirements.txt`.

Programación orientada a objetos

El proyecto **no exige** el uso de clases. Si el grupo decide modelar los productos o las ventas con clases, deberá mantener funcionando todos los criterios de aceptación y sustentar su decisión de diseño.

4. Criterios de aceptación

Para que el proyecto sea considerado aprobado, el programa debe cumplir los siguientes criterios.

4.1. Criterios funcionales

ID	Criterio
CF-01	Al iniciar, el programa carga el inventario desde `data/inventario.txt` y muestra el menú principal.
CF-02	El menú se repite hasta que el usuario elija la opción de salir.
CF-03	Se puede registrar un producto nuevo con todos sus atributos; el código duplicado es rechazado con un mensaje claro.
CF-04	Se pueden consultar productos por código y por categoría; si no existen, se informa sin que el programa falle.
CF-05	Se pueden modificar el precio y el stock de un producto existente, y eliminar un producto.
CF-06	Una venta admite varios productos; muestra el detalle (producto, cantidad, valor) y el total.
CF-07	Los descuentos del 5 % y 10 % se aplican correctamente según los umbrales definidos.
CF-08	El programa calcula el cambio y rechaza pagos insuficientes, permitiendo reintentar.
CF-09	Una venta que supere el stock disponible es rechazada, y el inventario no cambia.
CF-10	Al confirmar una venta, el stock se descuenta y la venta queda registrada en `data/ventas.txt`.
CF-11	El reporte de alertas lista los productos con stock menor o igual al mínimo, agrupados por proveedor, e identifica los agotados.
CF-12	El cierre del día muestra: total vendido, número de ventas, ticket promedio, producto más vendido y productos sin ventas.
CF-13	Al cerrar y reabrir el programa, el inventario y el historial se conservan.

4.2. Criterios de robustez

ID	Criterio
CR-01	Ingresar texto donde se espera un número no detiene el programa; se muestra un mensaje y se solicita el dato nuevamente.
CR-02	Los valores negativos o iguales a cero en precio, stock y cantidad son rechazados.
CR-03	Si `inventario.txt` no existe, el programa informa la situación y arranca con un inventario vacío (o lo crea).
CR-04	Si una línea del archivo está corrupta, se omite con una advertencia y se continúa con las demás.
CR-05	Los archivos se cierran correctamente aun cuando ocurra un error (uso de `finally` o `with`).
CR-06	Las violaciones a las reglas de negocio se señalan con `raise` y se capturan en el flujo principal.

5. Estructura sugerida del repositorio

```
tiendapy/
|-- README.md
|-- requirements.txt
|-- data/
|   |-- inventario.txt
|   `-- ventas.txt
|-- src/
|   |-- main.py          # menu principal y flujo general
|   |-- inventario.py    # CRUD de productos
|   |-- ventas.py        # carrito, descuentos y cambio
|   |-- reportes.py      # alertas y cierre del dia
|   |-- persistencia.py  # lectura y escritura de archivos
|   `-- validaciones.py  # captura segura de datos
```

La modularización en varios archivos es una sugerencia. Si el grupo prefiere un único archivo, las funciones deben estar claramente organizadas por secciones.

6. Entregables

1. **Repositorio GitHub** con el código fuente, los archivos de datos de ejemplo y el historial de commits completo.